LAPORAN PRAKTIKUM PEMROGAMAN AI
PENGGUNAAN FUNGSI (FUNCTION) DAN SCOPE VARIABEL

Disusun Oleh :

Nama : Tsabita Amalia Amzani

NIM : 2502030

Kelas : TI 3B

JURUSAN TEKNIK INFORMATIKA

POLITEKNIK PURBAYA

2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum pemrograman AI yang membahas mengenai topik Fungsi (Function) dan Scope Variabel ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk memenuhi tugas praktikum serta memberikan pemahaman mendalam mengenai konsep pendefinisian fungsi, pemanggilan fungsi dengan argumen posisional maupun *keyword arguments*, pengembalian nilai (*return value*), hingga lingkup variabel (*scope* lokal dan global) pada Python. Penulis berharap laporan ini dapat memberikan manfaat dan menjadi referensi pendukung bagi pembaca yang sedang mempelajari dasar-dasar pemrograman Python.

Penulis menyadari bahwa laporan ini masih memiliki keterbatasan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan karya-karya selanjutnya.

Margasari, 10 November 2026

Penulis

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 1
BAB II PROGAM DAN ANALISIS PROGRAM 2
2.1 Program 1: Deklarasi dan Pemanggilan Fungsi Sederhana tanpa Parameter 2
2.2 Program 2: Fungsi dengan Nilai Kembalian (*Return Value*) 2
2.3 Program 3: Fungsi dengan Parameter dan Input Pengguna 2
2.4 Program 4: Fungsi Perkalian 2 Bilangan dengan Pengembalian Nilai 3
2.5 Program 5: Pemanggilan Fungsi dengan *Keyword Arguments* (Argumen Bernama) 3
2.6 Program 6: Scope Lokal Variabel 3
2.7 Program 7: Pertukaran Variabel (Variable Swapping) di Dalam Fungsi 4
BAB III PENUTUP 5
3.1 Kesimpulan 5
3.2 Saran 5

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak, efisiensi dan struktur kode merupakan hal yang sangat krusial. Penulisan kode yang berulang (*redundant*) tidak hanya memperbesar ukuran program, tetapi juga menyulitkan proses perawatan (*maintenance*) dan pencarian kesalahan (*debugging*).

Bahasa pemrograman Python menyediakan fitur Fungsi (Function) sebagai blok kode terorganisir yang dapat digunakan kembali (*reusable*). Selain pemanggilan argumen secara posisional, Python juga mendukung pemanggilan fungsi menggunakan Keyword Arguments (Argumen Bernama), di mana nama parameter ditentukan secara eksplisit saat pemanggilan fungsi. Pemahaman terhadap pemanggilan argumen ini, serta pemisahan *scope* variabel (lokal vs global), sangat penting agar eksekusi program berjalan presisi sesuai logika yang diharapkan.

1.2 Tujuan

1. Memahami cara deklarasi dan pemanggilan fungsi sederhana tanpa parameter maupun dengan parameter.
2. Memahami penggunaan kata kunci return untuk mengembalikan nilai hasil komputasi fungsi.
3. Memahami interaksi input pengguna (*user input*) dengan fungsi pada Python.
4. Memahami penerapan pemanggilan fungsi menggunakan *Keyword Arguments* (argumen bernama).
5. Memahami konsep ruang lingkup variabel (*Scope Variable*: Lokal vs Global).
6. Memahami mekanisme pertukaran nilai variabel (*variable swapping*) di dalam lingkup fungsi dan efeknya terhadap variabel luar.

1.3 Manfaat

1. Mengasah pemahaman teknis mengenai logika pemrosesan fungsi di Python serta meningkatkan kemampuan dalam mendokumentasikan dan menganalisis kode program.
2. Menyediakan panduan referensi yang terstruktur mengenai penerapan fungsi di Python lengkap dengan analisis kasus berhasil maupun *error*.

BAB II
PROGAM DAN ANALISIS PROGRAM

2.1 Program 1: Deklarasi dan Pemanggilan Fungsi Sederhana tanpa Parameter

```
#deklarasi fungsi
def kalimat():
    "Menampilkan kalimat Hallo, Selamat Belajar Python"
    print('Hallo, Selamat Belajar Python')

#Program Utama
#memanggil fungsi
kalimat()
```

```
Hallo, Selamat Belajar Python
```

Fungsi kalimat() didefinisikan menggunakan kata kunci def. Di dalam fungsi terdapat *docstring* yang menjelaskan deskripsi fungsi dan instruksi print() untuk mencetak kalimat. Ketika kalimat() dipanggil pada program utama, instruksi cetak di dalam fungsi langsung dieksekusi.

2.2 Program 2: Fungsi dengan Nilai Kembalian (*Return Value*)

```
#fungsi hitung
def hitung():
    return 1+2

#panggil fungsi coba
tampil=hitung()
print('Hasil Perhitungan : ',tampil)
```

```
Hasil Perhitungan :  3
```

Fungsi hitung() melakukan penjumlahan 1+2 dan mengembalikan nilainya menggunakan perintah return. Di program utama, nilai kembalian ditampung ke dalam variabel tampil, lalu dicetak menggunakan print(), menghasilkan output 3.

2.3 Program 3: Fungsi dengan Parameter dan Input Pengguna

```
#deklarasi fungsi
def datadiri(nama,kota):
    "Menampilkan nama dan kota"
    print('Nama Anda : ',nama)
    print('Dari Kota : ',kota)

#Program Utama
nama=input('Masukkan nama Anda : ')
kota=input('Masukkan kota asal : ')

#memanggil fungsi
datadiri(nama,kota)
```

```
Masukkan nama Anda :  Tsabita
Masukkan kota asal :  Jakarta
Nama Anda :  Tsabita
Dari Kota :  Jakarta
```

Fungsi datadiri(nama, kota) menerima dua argumen. Program utama meminta input teks dari pengguna untuk variabel nama ("Tsabita") dan kota ("Jakarta"). Nilai-nilai tersebut kemudian dilewatkan sebagai argumen saat memanggil fungsi datadiri(nama, kota), yang mencetak data sesuai input.

2.4 Program 4: Fungsi Perkalian 2 Bilangan dengan Pengembalian Nilai

```
#deklarasi fungsi
def perkalian(bil1,bil2):
    "Menghitung perkalian 2 bilangan bulat"
    hasil = bil1 * bil2
    return hasil

#Program Utama
bil1=int(input('Input bil 1 : '))
bil2=int(input('Input bil 2 : '))
#memanggil fungsi
print('Hasil Perkalian ',bil1,' dengan ',bil2,' adalah ',perkalian(bil1,bil2))
```

```
Input bil 1 :  5
Input bil 2 :  6
Hasil Perkalian  5  dengan  6  adalah  30
```

Input pengguna dikonversi menjadi tipe data integer int(). Fungsi perkalian(bil1, bil2) menerima dua parameter numerik, mengalikan nilainya, dan mengembalikan hasil. Panggilan perkalian(bil1, bil2) dimasukkan secara langsung di dalam argumen print().

2.5 Program 5: Pemanggilan Fungsi dengan *Keyword Arguments* (Argumen Bernama)

```
def perkalian(bil1,bil2):
    "Menghitung perkalian 2 bilangan bulat"
    hasil=bil1*bil2
    return hasil

#Program Utama
bil1=int(input('Input bil 1 : '))
bil2=int(input('Input bil 2 : '))
#memanggil fungsi
print('Hasil Perkalian ',bil1,' dengan ',bil2,' adalah ', perkalian(bil2=bil1,bil1=bil2))
```

```
Input bil 1 :  5
Input bil 2 :  7
Hasil Perkalian  5  dengan  7  adalah  35
```

Program ini mendemonstrasikan penggunaan Keyword Arguments (argumen bernama) saat pemanggilan fungsi. Dengan menuliskan perkalian(bil2=bil1, bil1=bil2), nilai parameter diserahkan secara eksplisit berdasarkan nama parameternya tanpa terikat pada urutan posisi. Meskipun posisi parameter ditukar saat pemanggilan, hasil perhitungan perkalian tetap akurat dan berjalan dengan baik.

2.6 Program 6: Scope Lokal Variabel

```
def ContohLokal():
    A=99
    print("%d adal dalam Fungsi ContohLokal" %(A))
#Program Utama
A=55
print("%d adal dalam Fungsi ContohLokal" %(A))
ContohLokal()
```

```
55 adal dalam Fungsi ContohLokal
99 adal dalam Fungsi ContohLokal
```

Program ini menunjukkan perbedaan *scope* variabel. Variabel A = 55 di program utama merupakan variabel lokal, sehingga perintah cetak pertama menampilkan nilai 55. Sementara itu, A = 99 di dalam fungsi ContohLokal() adalah variabel lokal yang hanya berlaku di dalam lingkup fungsi tersebut. Saat ContohLokal() dipanggil, fungsi mencetak nilai lokal 99 tanpa mengubah nilai variabel lokal A.

2.7 Program 7: Pertukaran Variabel (Variable Swapping) di Dalam Fungsi

```
# fungsi mulai di sini
def Bilangan(x, y):
    print("Dalam fungsi:")
    print("Sebelum proses:")
    print("Nilai x", x)
    print("Nilai y", y)
    z = x
    x = y
    y = z
    print("Setelah proses:")
    print("Nilai x", x)
    print("Nilai y", y)

# program utama mulai di sini
x = 12
y = 3
print("Sebelum memanggil fungsi, x bernilai", x)
print("Sebelum memanggil fungsi, y bernilai", y)
Bilangan(x, y)
print("Setelah memanggil fungsi, x bernilai", x)
print("Setelah memanggil fungsi, y bernilai", y)
```

```
Sebelum memanggil fungsi, x bernilai 12
Sebelum memanggil fungsi, y bernilai 3
Dalam fungsi:
Sebelum proses:
Nilai x 12
Nilai y 3
Setelah proses:
Nilai x 3
Nilai y 12
Setelah memanggil fungsi, x bernilai 12
Setelah memanggil fungsi, y bernilai 3
```

Program ini menguji efek pertukaran nilai di dalam lingkup fungsi. Di dalam fungsi Bilangan(x, y), nilai parameter x dan y ditukar menggunakan variabel bantuan z. Sebelum proses nilainya adalah x=12, y=3, dan setelah proses menjadi x=3, y=12. Pada Python, argumen bertipe data imutabel (seperti integer) dilewatkan secara *pass-by-assignment*. Perubahan nilai x dan y di dalam fungsi bersifat lokal dan tidak memengaruhi nilai variabel x dan y pada lingkup program utama (yang tetap bernilai 12 dan 3).

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan serangkaian percobaan dan analisis praktikum yang telah dilakukan, dapat disimpulkan bahwa penggunaan Fungsi (*Function*) pada bahasa pemrograman Python sangat membantu dalam menciptakan modularitas kode serta memungkinkan penggunaan kembali blok perintah secara efisien, baik melalui fungsi sederhana tanpa parameter maupun fungsi yang memanfaatkan parameter dan instruksi pengembalian nilai (*return*). Selain itu, fleksibilitas pemanggilan fungsi dapat ditingkatkan melalui penerapan *Keyword Arguments*, yang memungkinkan penyerahan nilai parameter secara spesifik tanpa harus terikat secara kaku pada urutan posisinya.

Dari sisi ruang lingkup, terbukti bahwa variabel yang didefinisikan di dalam fungsi bersifat lokal dan terisolasi dari variabel global di luar fungsi meskipun memiliki kesamaan nama. Terakhir, mekanisme pengiriman argumen bertipe imutabel secara *pass-by-assignment* memastikan bahwa manipulasi atau pertukaran nilai variabel di dalam fungsi tidak merusak atau mengubah nilai variabel asli yang berada pada lingkup program utama.

3.2 Saran

Dalam rangka mengoptimalkan penulisan kode dan pemanfaatan fungsi di masa mendatang, disarankan agar pengembang senantiasa memanfaatkan *keyword arguments* pada fungsi yang memiliki parameter kompleks guna meningkatkan keterbacaan kode (*code readability*) serta menghindari kesalahan penempatan nilai argumen. Jika memang diperlukan adanya modifikasi nilai variabel luar secara langsung dari dalam fungsi, dianjurkan untuk menggunakan kata kunci global secara bijak atau menerapkan mekanisme pengembalian nilai lewat return. Selain itu, untuk keperluan pertukaran nilai variabel (*variable swapping*), sebaiknya mulai dibiasakan menggunakan fitur khas Python (*Pythonic*) seperti *tuple unpacking* yang jauh lebih ringkas dan efisien dibandingkan harus mendefinisikan variabel perantara tambahan.